基于卡尔曼滤波与 MPC 的自动驾驶掉头决策与多目标路径优化

摘　要

自动驾驶车辆在城市道路场景中实现安全、高效的掉头操作，是当前智能交通领域面临的关键技术挑战之一。车辆需同时应对障碍物、严格的几何约束以及乘坐舒适性要求，这使得掉头行为成为自动驾驶系统决策与规划能力的集中体现。本文针对这一复杂场景，构建了“环境感知-轨迹预测-风险决策-路径规划-实时控制”一体化模型，旨在实现安全性、效率性与舒适性的多目标协同优化，为提升自动驾驶车辆在城市环境中的通行提供可靠的理论依据与技术方案。

问题一中，提出了一种基于多模态轨迹预测与动态安全窗口评估的掉头时机决策模型。该模型通过卡尔曼滤波预测障碍物未来轨迹，综合考虑安全距离约束与碰撞时间（TTC）指标，量化“安全窗口”评价体系，得到“可掉头”决策的双重阈值（安全窗口>0.7，最小距离>2.5）。并通过卡尔曼滤波的过程噪声模拟障碍物±5%速度误差，进行不确定性分析，提升了模型的抗干扰能力和鲁棒性，最终输出“可掉头”或“等待”决策。

问题二中，设计了基于贝塞尔曲线与多目标优化的路径规划算法。采用参数化曲线——贝塞尔曲线描述车辆运行轨迹，确保路径的连续性和平滑性。运用层次分析法（AHP），采取专家打分的方式，科学确定多目标权重系数（安全距离 0.627、舒适性 0.264、总耗时 0.109），将多目标优化问题转化为加权单目标优化问题进行求解。优化后的路径满足最小转弯半径（≥5.5m）、横向加速度（≤1.2m/s²）等约束条件，实现了关键性能指标的显著提升，通过可视化得到其具体路径曲线。通过比较优化前后的数据，得出优化后最小安全距离提升至 6.09m，横向加速度波动降至 0.0236，总耗时控制在 40 秒内。

问题三中，建立了基于实时风险评估的分层响应与动态调整机制。提出了以 TTC 为核心的四级风险响应机制（高风险 TTC≤1.5s 紧急制动、中风险 1.5s<TTC≤3s 减速增大 TTC、低风险 3.0s<TTC≤10.0sMPC 微调路径、无风险 TTC>10.0s 保持原路径）。结合模型预测控制（MPC）算法，实现局部路径重规划、减速避让和紧急制动。仿真验证显示，该机制能有效应对突发障碍物干扰：在突发场景下，调整后路径仍满足所有约束，微调时总耗时仅增加 0.4%，急停时可避免碰撞。确保了系统的实时性与稳定性。

本文模型充分考虑了实际城市交通环境中的不确定性、动态性与多约束性，为自动驾驶车辆在复杂场景下的安全、高效、舒适掉头提供了系统的解决方案，对推动智能交通系统的实际部署具有参考意义。

关键词：自动驾驶　掉头决策　路径规划　卡尔曼滤波　多目标优化　MPC　动态调整

1　问题背景与重述

1.1　问题背景

目前，经济发展导致城市人口急剧膨胀，车辆数量迅速增加，这对城市道路交通产生了严重影响[1]。尽管交通管理措施和安全技术的进步使得我国交通事故的死亡率逐步下降，但机动车保有量的持续增长和交通流量的增加，依然使得每年因交通事故造成人员伤亡的数量维持在较高水平，据不完全统计，每年死亡人数大致在 5 万至 7 万之间[2]。

根据《交通强国建设纲要》的规划[3]，我国将全力以赴推动交通发展，建设安全有效的智能交通系统，这是实现交通强国的必然途径。自动驾驶（Autonomous Driving Technology）作为智能交通系统在国际上的一个热门课题[4]，是未来的发展趋势[5]。

除了提升交通安全外，劳动力结构变化、交通效率优化、能源使用转型以及智能出行需求的增长也共同推动着先进辅助驾驶系统（Advanced Driver Assistance Systems，ADAS）与自动驾驶系统（Automated Driving Systems，ADS）的发展[6]。利用先进的感知技术、定位技术、规划控制技术以及智能化的云平台，自动驾驶车辆（AV）可以实现自动驾驶，平台可以实时监测车辆的运动情况、驾驶员的意图以及感知周围的环境，从而为驾驶员提供更加精准的路线规划，大大提升了驾驶的安全性以及道路交通的效率[1]。

1.2　研究目的

自动驾驶车辆在道路行使的过程中会遇到各种各样的问题，包含变道、停车以及掉头等。其中，掉头是自动驾驶汽车行驶中最复杂且发生频率最高的过程。本研究以双向两车道为典型场景，旨在通过建立数学模型与设计算法的方式，解决自动驾驶汽车的掉头问题，其核心问题包括掉头时机决策、最优路径规划以及突发情况下的动态调整。通过上述研究，为自动驾驶车辆在城市复杂环境中的掉头操作提供可量化的、可验证的解决方案，推动智能交通系统在安全性、效率性与舒适性方面的协同优化。

1.3　问题重述

问题一：针对双向两车道的城市道路场景，要求建立 AV 的掉头决策模型。根据题目中给出的场景参数以及 AV 实时观测的障碍物状态数据，建立数学模型来对 AV 是否掉头进行决策。在建模过程中，需要根据题目中的核心约束条件量化掉头的可行性条件，以及在障碍物速度存在±5%误差情况下分析模型的抗干扰能力，提升模型的鲁棒性。

问题二：在问题一可掉头的条件下进行分析，需要解决路径规划问题。规划从 AV 进入掉头入口前 5m、速度为 20km/h 到掉头完成的完整路径并使用参数化曲线的方式进行描述。其次需要建立多目标优化模型，确定以“总耗时最小化”“横向加速度波动最小化”“与障碍物距离最大化”为目标的权重系数或多目标优化算法。

问题三：针对掉头过程中可能出现的突发情况，在满足约束条件的情况下，建立实时的动态调整机制，判断是否需要停止掉头或调整 AV 的掉头路径，确保 AV 在突发情况下保持稳定的操作性能。

2 问题分析

2.1 问题一的分析

问题一需要建立掉头时机决策模型，采用多模态预测与分层强化学习的决策方法，通过量化动态安全窗口来评估 AV 与障碍物的时空交互关系。首先，基于质点模型建二维坐标系，根据 AV 在坐标系中的初始位置、时间和速度构建物理方程；其次，构建融合预测系统捕捉障碍物轨迹，利用卡尔曼滤波实现高精度的短期轨迹预测；最后，整合不同时间尺度的预测结果得到预测轨迹。

在安全窗口量化方面，系统需要实时计算三个关键的安全约束指标：空间安全距离确保与行人保持 1.5 米以上、与其他车辆保持 2 米以上的缓冲区域；时间评估需要确保整个掉头操作在 60 秒内完成；同时需要满足全程横向加速度不超过 $1.2m/s^2$的舒适性阈值。综合约束条件和预测轨迹，建立车辆动力学方程，求出 AV 和障碍物之间的距离以及 AV 掉头时所需的时间作为安全阈值。并且运用蒙特卡洛模拟以及卡尔曼滤波过程的噪声误差，进行不确定性分析，加强模型的抗干扰能力，提高模型的鲁棒性。

2.2 问题二的分析

问题二需要解决路径规划问题，采用基于多目标优化的样条路径规划方法，旨在统筹安全性、效率性和舒适性三大核心目标。首先，采用层次分析法构建目标权重体系，邀请专家对总耗时、横向加速度波动和障碍物最小距离三项指标进行两两比较，建立判断矩阵后，计算特征向量，最终确定各目标的合理权重，从而将多目标优化问题转化为线性加权的单目标优化问题。然后，确定 AV 掉头过程的边界条件，即起点与终点，运用贝塞尔曲线描述掉头过程全路径。最后，采用序列二次规划算法求解，通过迭代求解二次规划子问题逐步逼近最优解，得到 AV 掉头过程的路径表述及可视化结果。

2.3 问题三的分析

问题三需要对突发障碍物做出高效决策，为了解决这一问题，本文设计了一套基于实时风险评估与分层响应机制的动态调整策略。首先，进行实时风险评估。通过计算 AV 与突发障碍物之间的碰撞时间，并结合相对距离、速度方向等信息，构建了一个综合风险评价指标，依据该指标划分高、中、低三个风险等级。其次，引入模型预测控制（MPC）进行局部轨迹重规划。根据不同风险等级，采用相应层次的 MPC 控制器，生成新局部路径或采取停车等措施规避风险。最后，设置分层响应机制，将风险以及 AV 的不同动作进行一一对应，解决掉头场景中不同风险等级的动态响应问题。从而全面提升掉头过程中的安全性与鲁棒性。

2.4 流程图

本研究流程图如图 2.1 所示。

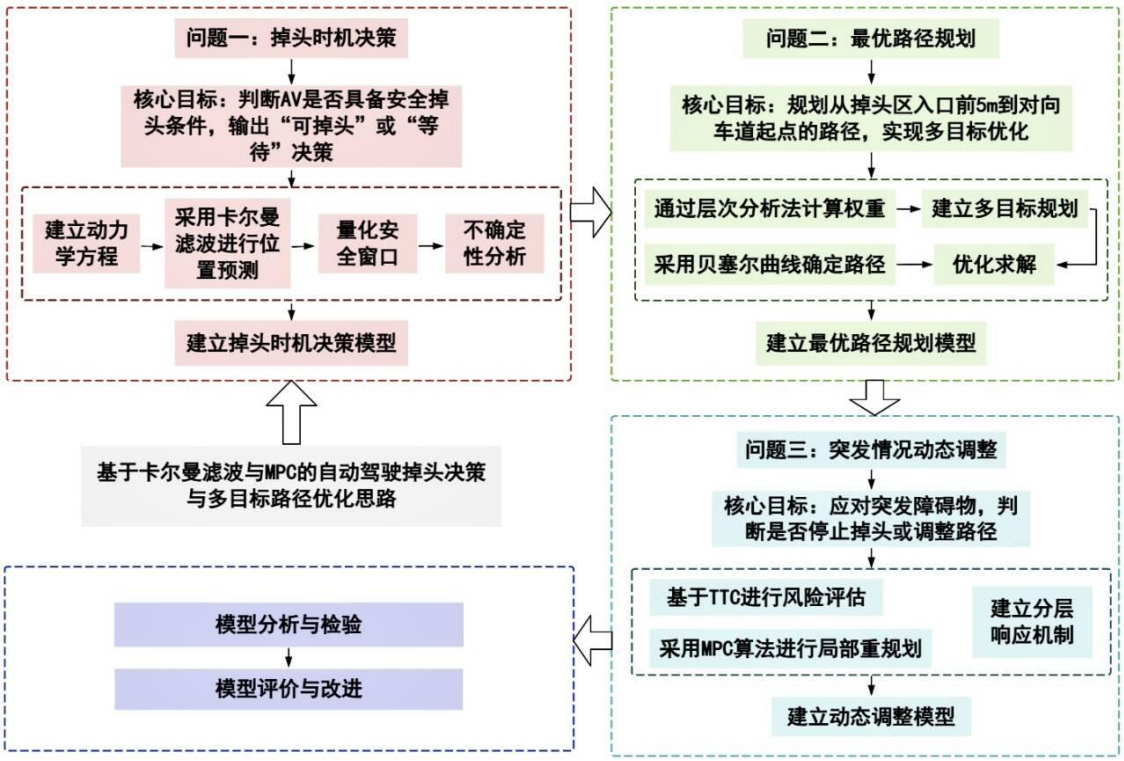

图 2.1 思路流程图

3 模型假设

1．为了简化模型，假设障碍物和 AV 的运动轨迹可以用质点模型描述，忽略车身尺寸对碰撞判断的影响。

2．假设障碍物的速度误差服从±5%的正态分布。

3．假设道路边界和掉头区为刚性约束，不考虑弹性变形。

4．假设自动驾驶车辆和感知系统在研究过程中不受其他因素影响，功能正常。

5．道路为理想平面，忽略路面坡度、颠簸等地形因素的影响。

6．不考虑交通信号灯、交通警察指挥等外部控制信号，AV 仅依据自身感知数据决策。

7．假设社会车辆的速度服从[20,40]km/h 的均匀分布，行人或非机动车横穿掉头区的方向随机。

4 符号说明

本研究涉及到的符号及其说明如表 4.1 所示。

表 4.1 符号说明

符号	说明	单位
x,y	车辆 / 障碍物的纵、横向位置坐标	m
v	速度	m/s
θ	航向角（与纵向轴线夹角）	rad

续表 4.1

符号	说明	单位
a	加速度	m/s^2
R	转弯半径	m
κ	路径曲率	$1/m$
$d_{\min}$	最小安全距离（行人 1.5m，车辆 2m）	m
TTC	碰撞时间（距离 / 相对速度）	s
J_1	总耗时目标函数	s
J_2	横向加速度波动目标函数	$(m/s^2)^2$
J_3	与障碍物距离目标函数	m
$\omega_1,\omega_2,\omega_3$	多目标优化权重	-

5 模型的建立与求解

5.1 问题一模型的建立与求解

5.1.1 模型建立

Step1 建立车辆运动学方程

将车道看作平面坐标系，用 x 和 y 来表示 AV 和障碍物的横纵坐标，进而表示车辆的位置，以便于对车辆位置更新的分析及对距离的判断。

根据物理学知识，得到位置更新公式为：

$$\begin{cases} x_{k+1} = x_k + v_k \cdot \cos(\theta_k) \cdot \Delta t \\ y_{k+1} = y_k + v_k \cdot \sin(\theta_k) \cdot \Delta t \end{cases} \tag{5.1}$$

其中，x_k、y_k 为 k 时刻的车辆纵横向位置的坐标，v_k 为 k 时刻车辆的速度，θ_k 为 k 时刻车辆的航向角，Δt 为时间步长，在本文中为 0.1 秒。

得到速度更新公式为：

$$v_{k+1} = v_k + a \cdot \Delta t \tag{5.2}$$

其中，a 为加速度，对于 AV 来说受限于最大纵向加速度以及最大横向加速度。

同时，当 AV 进行掉头过程时受到几何约束，其路径须满足最小转弯半径 5.5m。为满足该约束条件，建立判断最小转弯半径约束的方程如下：

$$R_{\min} \geqslant \frac{1}{\max|\kappa|} \tag{5.3}$$

其中，κ 为路径曲率。

通过上述操作得到车辆行驶的运动学方程，便于量化约束条件，建立安全评估模型，获得安全阈值。

Step2 建立卡尔曼滤波短期预测模型

卡尔曼滤波又被称为线性状态估计器，能够根据物体上一帧状态信息(即前景灰度区域信息)以及均方差通过递归模型对物体当前帧的状态进行最优估计，其主要包括预测和更新两个步骤[7]。为了对 AV 以及障碍物的位置信息进行预测，使用卡尔曼滤波预测模型，得到其位置变化的信息，以便于对掉头期间是否发生碰撞的安全性约束进行量化。

首先，根据运动学方程中的变量定义其状态向量：

$$X_k = [x_k, y_k, v_k, \theta_k, a_k]^T \tag{5.4}$$

然后，根据已有的卡尔曼滤波建立状态预测方程，对该状态向量进行预测：

$$X_k = F_k X_{k-1} + W_k \tag{5.5}$$

其中，W_k 为状态转移过程产生的过程误差，将其称为过程噪声，描述了过程中产生的不确定性。

接着，忽略状态方程中的过程噪声并根据位置更新公式以及状态方程，建立两个时间点之间的微分方程式子，以此得出状态转移矩阵：

$$F = \begin{bmatrix} 1 & 0 & \Delta t\cos\theta_k & 0 & \frac{1}{2}\Delta t^2\cos\theta_k \\ 0 & 1 & \Delta t\sin\theta_k & 0 & \frac{1}{2}\Delta t^2\sin\theta_k \\ 0 & 0 & 1 & 0 & \Delta t \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix} \tag{5.6}$$

其中，由题目中 AV 的实时感知频率为$10HZ$，可得到其$\Delta t = 0.1s$。

最后，进行预测。在卡尔曼滤波中，一般将过程噪声看作标准正态分布，其协方差矩阵为$Q_k = \mathrm{diag}([0.1, 0.1, 0.05, 0.05, 0.01])$。在预测过程中，先设置初始值，然后运用该模型得到掉头过程中 AV 和障碍物的状态。

Step3　建立安全评估模型

根据题目中给出的安全性约束，可先进行数值化处理，得到安全距离的数学表达式为：

$$d_{safe}(i) = \begin{cases} 1.5 & \text{行人} \\ 2.0 & \text{社会车辆} \end{cases} \tag{5.7}$$

将平面坐标的两点距离作为该车辆掉头过程中与障碍物的最小距离，结合上述给出的位置信息得到每个时间点的距离公式，并比较得到其最小的距离作为其最小距离：

$$d_{\min} = \min\sqrt{(x_{av} - x_{obs})^2 + (y_{av} - y_{obs})^2} \tag{5.8}$$

其主要反映 AV 与周围障碍物的最近距离，给定安全评估的基础指标。

此外，进一步考虑不确定性的情况，即当进行卡尔曼滤波预测的时候会存在误差，将该误差考虑进来后得到公式：

$$d_{safe} = \max\left(0, d_{\min} - \sqrt{uncertainty}\right) \tag{5.9}$$

不确定性距离主要由卡尔曼滤波中的噪声体现，因该不确定距离由噪声的协方差表示。

为进一步保障安全性约束，引入碰撞时间指标，表示两车相向运动是的碰撞风险，

确保在存在感知或预测误差的情况下仍有足够的安全余量。其值为最小距离与两车相对速度的比值，公式如下：

$$TTC = \frac{d_{\min}}{v_{rel}}, if\ v_{rel} > 0 \tag{5.10}$$

其中，$v_{rel} = v_{av} - v_{obs}$ 为相对速度。TTC 越小，碰撞风险就越高，更能体现动态碰撞的风险，更有利于判断是否满足安全性约束。

将最小距离与碰撞时间通过公式进行结合，形成空间维度的安全阈值，其公式为：

$$safetyWindow = \min\left(1, \frac{\bar{d}_{\min}}{5}\right) \cdot \min\left(1, \frac{TTC_{\min}}{10}\right) \tag{5.11}$$

其中，$\bar{d}_{\min}$ 为平均最小距离，$TTC_{\min}$ 为最小碰撞时间，并且进行归一化处理确保安全窗口值在[0,1]范围内。

根据效率约束可知，从开始减速到掉头再到恢复直行的时间应不超过 60 秒。为进一步的去满足该条件，应通过公式去计算 AV 掉头过程的时间并与约束条件进行比较，将其看作判断是否可以进行掉头的阈值之一。

对于 AV 的掉头过程，主要分为三个阶段：

减速阶段：从起始速度 v_0 减速到掉头开始时的速度 v_1，耗时

$$t_1 = \frac{(v_0 - v_1)}{a_{纵}} \quad (a_{纵} \leqslant 2.5 m/s^2) \tag{5.12}$$

转向阶段：沿最小转弯半径路径完成转向的耗时

$$t_2 = \frac{\pi \cdot R}{v_1} \quad (R = 5.5m, v_1 为转向速度) \tag{5.13}$$

加速阶段：转向结束后的速度重新加速为起始速度的耗时

$$t_3 = \frac{(v_0 - v_1)}{a_{纵}} \tag{5.14}$$

将三个阶段的时间进行相加，得到整体过程的耗时，及需要满足的约束：

$$T = t_1 + t_2 + t_3，\ s.t.\ T \leqslant 60s \tag{5.15}$$

将其作为其中的一个阈值。

通过同时满足安全性约束以及效率约束，来进行构建安全窗口，通过算法得到满足掉头条件的时间和空间上的安全阈值。

Step4　不确定分析

题目中给出障碍物的速度存在 $\pm 5\%$ 的误差，因此需要考虑模型的不确定性。在进行卡尔曼滤波预测的时候会存在预测的误差情况，其中通过过程噪声来进行表述，因此障碍物的速度误差可表现在卡尔曼滤波的过程噪声中。过程噪声与速度误差可进行定量评估。

假设障碍物速度为 v，5%的速度误差对应的标准差为 $\sigma_v = 0.05v$，方差为 $\sigma^2{}_v = 0.0025v^2$。将其作为过程噪声的协方差，并进一步作为距离预测的不确定指标进行安全距离修正。通过该方法可以更好地提高模型的抗干扰能力。

此外，还可以通过蒙特卡洛模拟的方法生成 1000 组速度样本，并将障碍物速度视为随机变量$v \sim \mathbb{U}(v_{测} \times 0.95，v_{测} \times 1.05)$，重复预测步骤和距离计算，得到“安全窗口存在的概率”P。当$P \geqslant 95\%$是，输出“可掉头”，否则“等待”。

5.1.2 模型求解

1．参数初始化

对于场景参数，按赛题 B 设定双向两车道总宽 7m（单车道 3.5m），掉头区长度 20m，观测范围前后 50m，限速 40km/h（即 11.11m/s）；

对于 AV 参数，设定初始位置为掉头区入口前 5m（x=45m，y=1.75m），初始速度 20km/h（5.56m/s），最小转弯半径 5.5m，安全距离行人 1.5m、社会车辆 2m；

对于障碍物参数，随机生成 5 个障碍物（1-2 类为行人/非机动车，3-4 类为社会车辆），社会车辆速度服从[20,40]km/h 均匀分布（5.56-11.11m/s），行人速度满足再[1.5,5]km/h（0.42-1.39m/s）之间，横向或纵向横穿概率各 50%，出现概率 0.3/10 秒。

2．卡尔曼滤波轨迹预测实现

对每个障碍物，基于前 10 步历史轨迹（位置、速度、航向），通过卡尔曼滤波预测未来 5 秒（50 步）轨迹，状态向量为模型建立时的X_k，状态转移方程为F以及其过程噪声和协方差。输出预测轨迹以及不确定性即协方差矩阵迹的平方根。

3．安全窗口计算与执行决策

基于预测的结果计算最小距离以及碰撞时间，将二者指标进行结合计算安全窗口（$safetyWindow$）值并及逆行归一化处理，根据代码最终计算得安全窗口值为 0.7。当输出的安全窗口值大于 0.7 时且最小距离大于 2.5 米时才允许执行掉头的决策。因此可得出衡量是否可掉头的阈值为安全窗口和最小距离，且其值分别要大于 0.7 和 2.5。

4．仿真结果分析

对预测的结果以及安全阈值进行可视化，得到包含三个关键部分的可视化结果：场景可视化，显示道路、掉头去、车辆轨迹和障碍物运动以及随时间变化的掉头决策和安全指标随时间的变化及决策阈值。部分可视化结果如图 5.1 和图 5.2 所示。

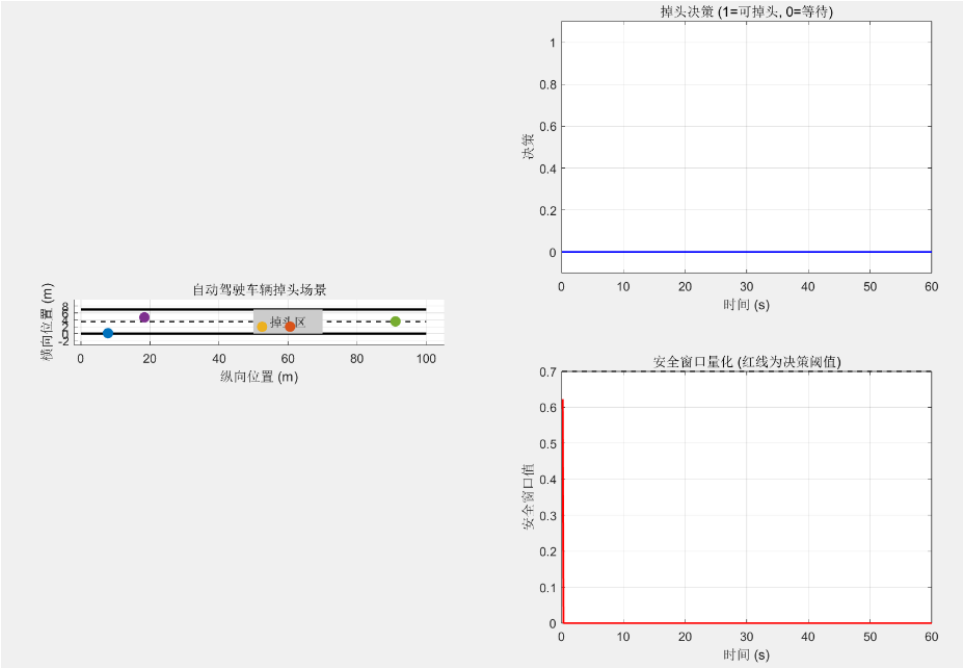

图 5.1 仿真一是否可掉头决策

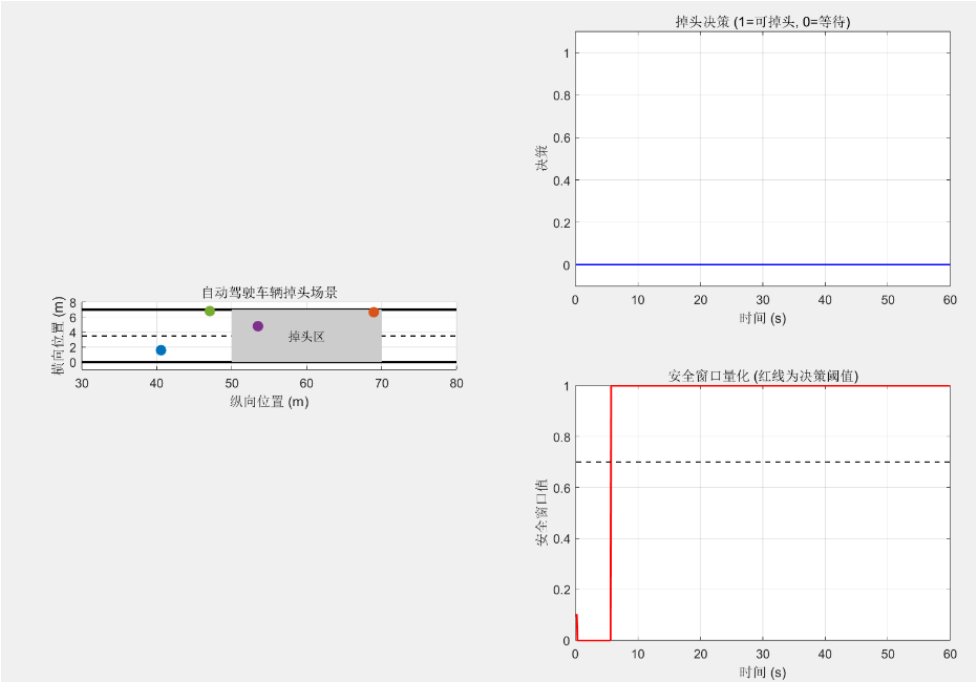

图 5.2 仿真二是否可掉头决策

对于图 5.1，存在对向车辆（蓝色圆点）位于纵向 40m、横向 2m 处，对向车辆导致最小距离过小（1.2m<安全距离 2m），TTC 极短（0.058s），安全窗口≈0，未达到 0.7 阈值。

对于图 5.2，无明显障碍物，但安全窗口阈值设为 1.0（红色水平线）。即使安全窗口值为 0.85（良好状态），但由于阈值设置过高（1.0），仍无法触发掉头决策，体现了模型的可靠性与安全性。

两张图片通过对比展示了安全窗口阈值和场景布局对掉头决策的影响，验证了代码中安全评估模型的逻辑正确性：决策一致性，安全窗口未达标时，决策始终为 0（等待），与代码逻辑一致。场景适应性，障碍物存在和阈值设置是影响决策的两大关键因素。安全保守性，系统设计遵循"宁可等待，不可冒险"原则，确保自动驾驶安全性。工程应用中，建议采用动态阈值调整策略，平衡安全性和行驶效率，避免图片 2 中过度保守的决策行为。

5.2 问题二模型的建立与求解

5.2.1 模型建立

Step1 定义目标函数

为了对 AV 在特定位置掉头的完整路径进行多目标优化，需要分别定义“总耗时”、“横向加速度波动”和“与障碍物距离”所代表的数学表达式，通过赋权的方式将多目标规划转化为单目标规划。

首先，对三个目标进行数学描述：“总耗时”即 AV 在进行掉头过程中总共消耗的时间，在问题一中已进行了详细的表述，用字母T表示。

“横向加速度的波动”即 AV 行驶过程中横向加速度的变化。横向加速度为横向位置对时间的二阶导数，横向加速度的波动则为加速度随时间的变化率。因此采用积分平方形式量化波动，其公式为：

$$\int_0^T \left(\ddot{y}(t)\right)^2 dt \tag{5.16}$$

“与障碍物距离”即在车辆掉头过程中与周围障碍物的距离，在问题一中也已进行了详细表述，令其为$d_{\min}$。

然后，明确多目标规划要求三个目标的最小化，根据其具体要求建立相应的目标函数。对于“总耗时最小化”来说，要求使总耗时最小则可建立目标函数：

$$J_1 = T \tag{5.2}$$

对于“横向加速度波动最小化”来说，可建立目标函数：

$$J_2 = \int_0^T \left(\ddot{y}(t)\right)^2 dt \tag{5.2}$$

对于“与障碍物距离最大化”来说，由于其为最大化问题，因此要对目标函数取负号。可建立目标函数：

$$J_3 = -d_{\min} \tag{5.2}$$

最后，需要将三个目标函数进行线性组合并设置权重来将多目标优化转化为单目标优化，权重系数通过层次分析法来确定。

Step2　层次分析法确定权重系数

由步骤一可得出三个目标函数，现对目标函数进行线性加权，分别设置它们的权重为ω_1、ω_2、ω_3。则通过线性加权得到的单目标函数为：

$$\min\ J = \omega_1 J_1 + \omega_2 J_2 + \omega_3 J_3 \tag{5.2}$$

其中权重系数满足$\omega_1 + \omega_2 + \omega_3 = 1$。为确定具体权重，将采用层次分析法。

层次分析法（Analytic Hierarchy Process,AHP）是由美国运筹学家托马斯·L·萨蒂于 20 世纪 70 年代提出的一种定性与定量相结合的系统决策分析方法。AHP 通过构建层次结构模型，将复杂问题分解为若干层次，并对各层次中的因素进行比较、判断和计算，从而得出各因素的相对重要性权重，为决策提供依据[8]。

其主要包括三个主要层面：目标层、准则层以及方案层。在该问题中目标层为最优路径规划，准则层为“总耗时最小化”、“横向加速度波动最小化”、“与障碍物距离最大化”三个方面。

首先，采取专家打分的方式，以安全优先为原则，构建准则层的 pairwise 比较矩阵，其矩阵构造如表 5.1 所示。

表 5.1 pairwise 比较矩阵

准测	C1（总耗时）	C2（舒适性）	C3（安全距离）
C1	1	1/3	1/5
C2	3	1	1/3
C3	5	3	1

其中，舒适性即指横向加速度波动最小化，安全距离即指与障碍物距离最大化。上述

标度分别表示为：1-同等重要，3-前者比后者重要，5-前者比后者明显更重要。通过比较性质的度量可以将每个目标函数的重要性量化，为权重计算提供数据，确保决策的科学性和一致性，最终支持多目标优化中的权重分配。

其次，进行权重计算，采用几何平均法，其公式为：

$$GM_i = \sqrt[n]{\prod_{j=1}^{n} A_{ij}} \tag{5.21}$$

即对 pairwise 比较矩阵的每行元素求乘积再开 n 次方根，

然后，进行归一化处理，将几何平均转化为权重向量，确保权重和为 1：

$$\omega_i = \frac{GM_i}{\sum GM_i} \tag{5.22}$$

最后，进行一致性检验，其作用在于验证比较矩阵的一致逻辑性。一致性检验一般分为三部分，计算 pairwise 比较矩阵最大特征值、一致性指标和随机一致性比率，若满足条件 CR<0.1，则通过一致性检验权重合理，若不满足条件则需要通过重新构造矩阵。对于一致性指标，其计算公式为：

$$CI = \frac{\lambda_{\max} - n}{n-1} \qquad (5.2)$$

其中，n 表示为矩阵的阶数。

对于随机一致性指标，其计算公式为：

$$CR = \frac{CI}{RI} \qquad (5.2)$$

其中，RI 为随机一致性指标，通过随机生成大量同阶矩阵统计得到。

Step3　建立多目标路径规划模型

运用贝塞尔曲线生成 AV 掉头过程的路径，其公式为：

$$P(t) = (1-t)^3 P_0 + 3(1-t)^3 t P_1 + 3(1-t) t^2 P_2 + t^3 P_3 \qquad (5.2)$$

其中，$t \in [0,1]$，$P_0 - P_3$ 为控制点。通过控制包括起点与终点在内的 4 个点生成 AV 掉头过程的路径，并通过调整中间的控制点改变路径形状。在得到路径之后，通过计算曲率以及横向加速度对路径进行评估，其中曲率以及横向加速度的计算在上述内容中已详细表述。

生成掉头路径后，需要根据题目中已给出的条件对该路径进行多目标优化。建立多目标优化模型，其中的目标函数为上述通过线性加权得到的单目标函数。其约束条件满足题目中给出的约束条件以及要求。最终完成路径的优化过程。

5.2.2　模型求解

1．权重分析结果

使用 AHP 计算三个目标函数的权重系数，使用 MATLAB 代码计算权重值，得到权重向量为 $[0.1047, 0.2583, 0.637]$。因此可得到线性加权后的单目标函数为：

$$\min\ J = 0.1047 J_1 + 0.2583 J_2 + 0.637 J_3 \tag{5.26}$$

由该权重关系可知，与障碍物的距离在多目标优化的过程中占比最大即表明安全性在掉头过程中最为重要，掉头时驾驶人员的舒适性次之，其掉头的效率在目标中的占比

最少。

得到权重之后要对其进行一致性检验，根据公式计算得到最大特征值、一致性指标以及一致性比率，如表 5.2 所示。

表 5.2 一致性检验结果

最大特征值	一致性指标	一致性比率
3.0385	0.0193	0.0332

其中，一致性比率为 0.00332 小于 0.1，则通过一致性检验。

2．路径模拟结果

运用贝塞尔曲线进行规划路径，给定初始条件初始位置为掉头区入口前 5m（x=45m，y=1.75m），初始速度 20km/h（5.56m/s）。根据 MATLAB 代码计算得到 AV 掉头过程中的路径变化参数如表 5.3 所示，限于篇幅只展示部分数据，其余数据在附录中。

表 5.3 掉头过程路径参数

时间(s)	x(m)	y(m)	速度(m/s)	转向角(°)	旋转角(°)	横向加速度(m/s²)
0	-5	1.8	5.56	-3.9	-3.9	0.36
0.1	-3.8	1.7	5.75	-3.8	-3.8	0.53
0.2	-2.6	1.6	5.94	-3.7	-3.7	0.7
0.2	-1.5	1.5	6.14	-3.5	-3.5	0.69
0.3	-0.3	1.5	6.33	-3.4	-3.4	0.68
0.4	0.8	1.4	6.53	-3.2	-3.2	0.67
0.5	1.8	1.3	6.72	-3.1	-3.1	0.66
0.5	2.9	1.3	6.92	-2.9	-2.9	0.65
0.6	3.9	1.2	7.11	-2.8	-2.8	0.64
0.7	5	1.2	7.3	-2.6	-2.6	0.63
0.8	6	1.1	7.5	-2.5	-2.5	0.62
0.9	6.9	1.1	7.69	-2.3	-2.3	0.61
0.9	7.9	1.1	7.89	-2.1	-2.1	0.6
1	8.8	1	8.08	-1.9	-1.9	0.59
1.1	9.7	1	8.27	-1.7	-1.7	0.58
1.2	10.6	1	8.47	-1.6	-1.6	0.57

对该参数进行可视化，得到 AV 掉头过程的路径曲线，如图 5.3 所示。

对是否满足约束条件进行分析，结果如表 5.4 所示。根据表 5.4 以及题目已给的条件可知，满足约束条件。

表 5.4 约束条件分析结果

指标	数值
总耗时(s)	7.77
横向加速度波动	0.2801
最小安全距离(m)	2.16
计算时间(s)	0.001
约束满足率(%)	85.7

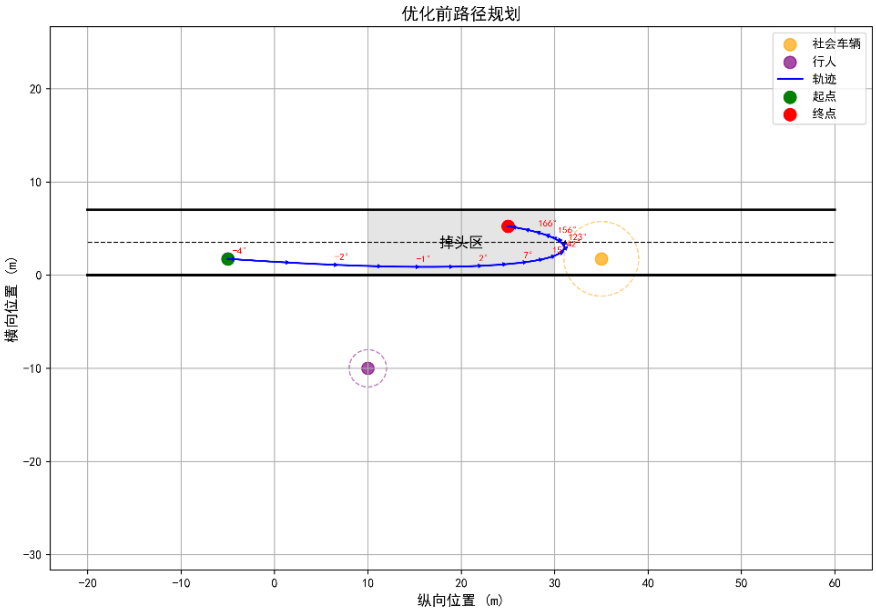

图 5.3 AV 掉头路径曲线

3．多目标优化后的路径变化

为了满足给定的三个目标函数，建立多目标优化模型，一线性加权的单目标函数作为目标函数以给出的约束条件为目标规划中的约束条件，重新根据贝塞尔曲线生成 AV 掉头过程中的路径参数如表 5.5 所示（仅展示部分参数）。

表 5.5 优化后的 AV 掉头过程路径参数

时间(s)	x(m)	y(m)	速度(m/s)	转向角(°)	旋转角(°)	横向加速度(m/s²)
0	-5	1.8	5.34	20.8	20.8	-0.02
0.4	-4.7	1.9	4.34	19.9	19.9	-0.03
0.8	-4.4	2	3.94	18.3	18.3	-0.04
1.2	-4	2.1	4.15	16.8	16.8	-0.04
1.6	-3.7	2.2	4.37	15.5	15.5	-0.04
2	-3.3	2.3	4.59	14.2	14.2	-0.03
2.4	-3	2.4	4.82	13	13	-0.03
2.8	-2.6	2.5	5.05	11.9	11.9	-0.03
3.2	-2.2	2.5	5.29	10.9	0.9	-0.03
3.6	-1.8	2.6	5.53	10	10	-0.03
4	-1.4	2.7	5.78	9.1	9.1	-0.03

对结果进行可视化，得到 AV 掉头时的路径曲线，如图 5.5 所示。

且对是否满足约束条件进行分析，可得到如表 5.6 所示。

表 5.6 优化后的约束条件分析结果

指标	数值
总耗时(s)	40
横向加速度波动	0.0236
最小安全距离(m)	6.09
计算时间(s)	4.0306
约束满足率(%)	85.7

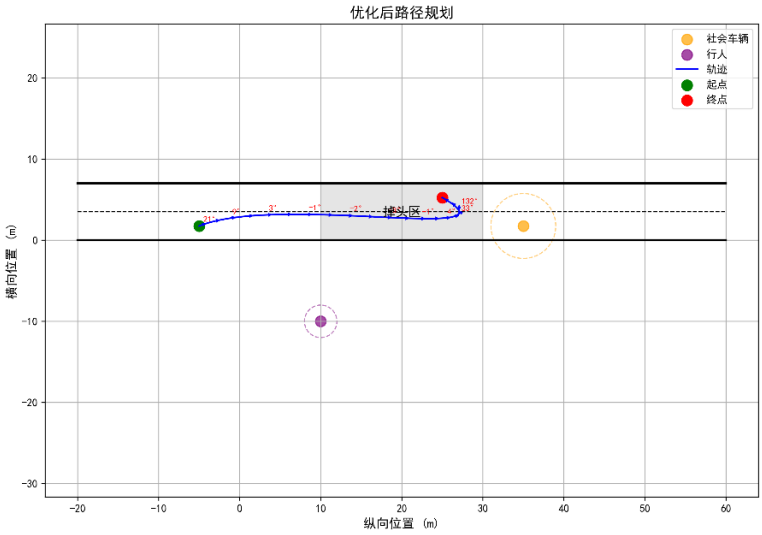

图 5.5 优化后的 AV 掉头路径曲线

根据表 5.6 以及题目已给的条件可知其满足约束条件。且对优化前与优化后的指标进行对比可知，优化后的最小距离大于优化前的最小距离，可以看出优化后距离更大表明在进行掉头过程中更加安全。此外优化后的横向加速度波动小于优化前的结果，且几乎趋近于 0，可知优化后在掉头过程中驾驶的舒适性更强。但是可看出优化后的耗时明显高于优化前的耗时，则可知在增大安全性和舒适性的同时牺牲了对于效率的追求，同时也体现了多目标优化时的权重，以及各自过程中的重要性。

5.3 问题三模型的建立与求解

5.3.1 模型建立

Step1 风险评估

掉头过程中若出现突发情况，如突然有行人/非机动车闯入掉头区域，此时需要对 AV 的掉头过程进行调整，同停止掉头或调整掉头路径来避免完全事故的发生。因此，需要考虑的问题是在什么情况下需要对 AV 进行调整。为了解决这一问题，我们引入碰撞时间（TTC）这一指标进行风险评估。

TTC（碰撞时间）是衡量碰撞紧迫性的关键指标，通过当前距离与相对速度的比值预测碰撞发生时间，值越小表明碰撞风险越高。基于 TTC 值制定三级风险响应机制，实现风险的量化评估与分级处理。

TTC 的计算公式如下所示：

$$TTC = \frac{current_{distance}}{relative_{speed}} \tag{5.27}$$

其中，$current_{distance}$ 为车辆障碍物的直线距离，$relative_{speed}$ 为车辆与障碍物的相对速度，

同向行驶时为 $v_{rel} = |v_{av} - v_{obs}|$，相向行驶时 $v_{rel} = v_{av} + v_{obs}$。

依据计算出的碰撞时间划分风险等级：

Level3（高风险）：$TTC \leqslant 1.5s$ 或距离小于安全距离，则进行紧急制动；

Level2（中风险）：$1.5s < TTC \leqslant 3.0s$，则减速增大 TTC；

Level1（低风险）：$3.0s < TTC \leqslant 10.0s$，则 MPC 微调路径；

Level0（无风险）：$TTC > 10.0s$，则维持原来路径。

划分好风险等级后，需根据安全触发条件进行判断即通过问题一得到的安全阈值进行判断。

Step2　局部重规划算法

在确定了风险等级后，需要对不同的情况进行判断，并根据判断的结果进行优化和调整。采用模型预测控制（MPC）算法，MPC 算法是通过“预测-优化-反馈”的方式进行的，其核心过程为：首先，运用简化的自行车模型来预测车辆的未来运动，其公式如下：

$$\begin{cases} \dot{x} = v\cos(\theta+\beta) \\ \dot{y} = v\sin(\theta+\beta) \\ \dot{\theta} = \dfrac{v}{L}\tan(\delta) \\ \beta = \arctan\left(\dfrac{l_r}{L}\tan(\delta)\right) \end{cases} \tag{5.28}$$

其中，β 为侧偏向角，l_r 为后轴到质心距离，L 为轴距，δ 为前轮转向角，其余的符号与上述问题中的表述相同，在具体的算法中将会进行适当改变。

其次，预测出具体行动后，根据 TTC 与障碍物的距离共同判断风险等级并且做出相应的优化调整，并生成优化控制指令。同时根据障碍物横向距离动态调整转向增益，避免过度转向导致的不舒适。MPC 的优化问题的数学公式如下所示：

$$\min_{u} \ J = \sum_{k=0}^{N-1}\left[e(k)^T Q e(k) + u(k)^T R u(k)\right] + e(N)^T P e(N) \tag{5.29}$$

其中，$e(k)$ 为跟踪误差，$u(k)$ 为控制输入（转向角、加速度），Q 为状态误差权重，R 为控制量权重，P 为终端误差权重。

最后，以公式 5.29 作为该优化问题的目标函数，以题目中给出的约束条件作为该优化问题的约束条件，以此对掉头过程进行微调。

该算法通过先预测未来短时间内的轨迹，再求解带约束的二次规划问题得到控制序列，最后再执行该指令，并且不断重复该过程，以此来实现对路径的局部调整。

Step3　建立分层响应机制

完成对突发问题进行了风险评估，并且运用 MPC 算法对掉头路径进行局部重规划后，将二者进行一一对应，即判断在什么风险等级要进行怎样的路径调整。为了解决这一问题，运用分层响应机制进行判断，即根据上述划分出的风险等级以及分别对应的调整方式，安全、高效应对突发情况。

5.3.2　模型求解

1．对于微调路径的情况：

首先初始化相应相应基本参数，并对障碍物位置进行实时更新。根据问题二已预

测的 AV 车辆掉头路径以及其位置参数的变化，计算 AV 和障碍物的碰撞时间 TTC 和距离，并对应相应的风险等级。由代码计算可知，TTC 处于低风险阶段，因此运用 MPC 控制器进行微调，其调整路径曲线如图 5.6 所示。

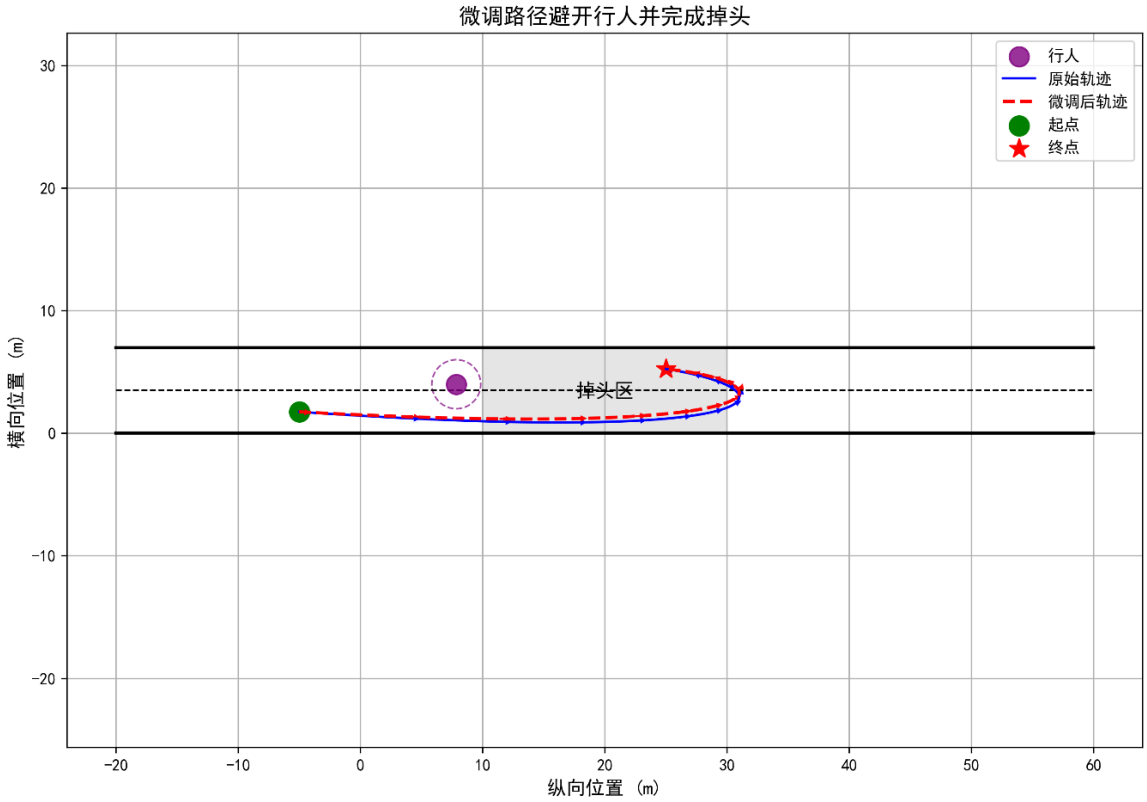

图 5.6 微调情况路径曲线变化

根据图 5.6 可以看出，其掉头路径发生了局部的变化，避免了在掉头结束后与行人相撞的情况，但其整体行驶轨迹依旧保持原掉头曲线。为进一步比较进行细微调整后路径变化，给出微调指标表，如表 5.7 所示：

表 5.7 微调情况指标结果

指标	原始值	调整后值	变化率(%)
平均横向加速度(m/s²)	0.22	0.167	-24.4
最大横向加速度(m/s²)	0.701	0.65	-7.3
平均速度(m/s)	4.7	4.69	-0.3
平均曲率(1/m)	0.273	0.2961	8.5
总耗时(s)	7.77	7.74	-0.4

由表 5.7 中数据可得，其平均横向加速度与最大横向加速度均较于原来的有所减小，可知 AV 掉头过程的舒适性有所提高。且其总耗时用时减小，其效率也有所提高。对于平均曲率而言其数值相较于原来有所提高，使掉头时的半径更大，更不容易与行人相撞。但所有指标的变化均在一个较小的范围内，保证了路径的局部微调。因此，在遇到突发状况时，当 TTC 处于低风险时，将进行细微的路径调整以确保不发生危险事故，同时保证了驾驶时的舒适性与效率。

2．对于急停情况：

障碍物路径更新时若与 AV 掉头过程所计算的 TTC 位于高风险时，将采取急停措施，代码运行过程中出现急停状况的路径图如图 5.7 所示。

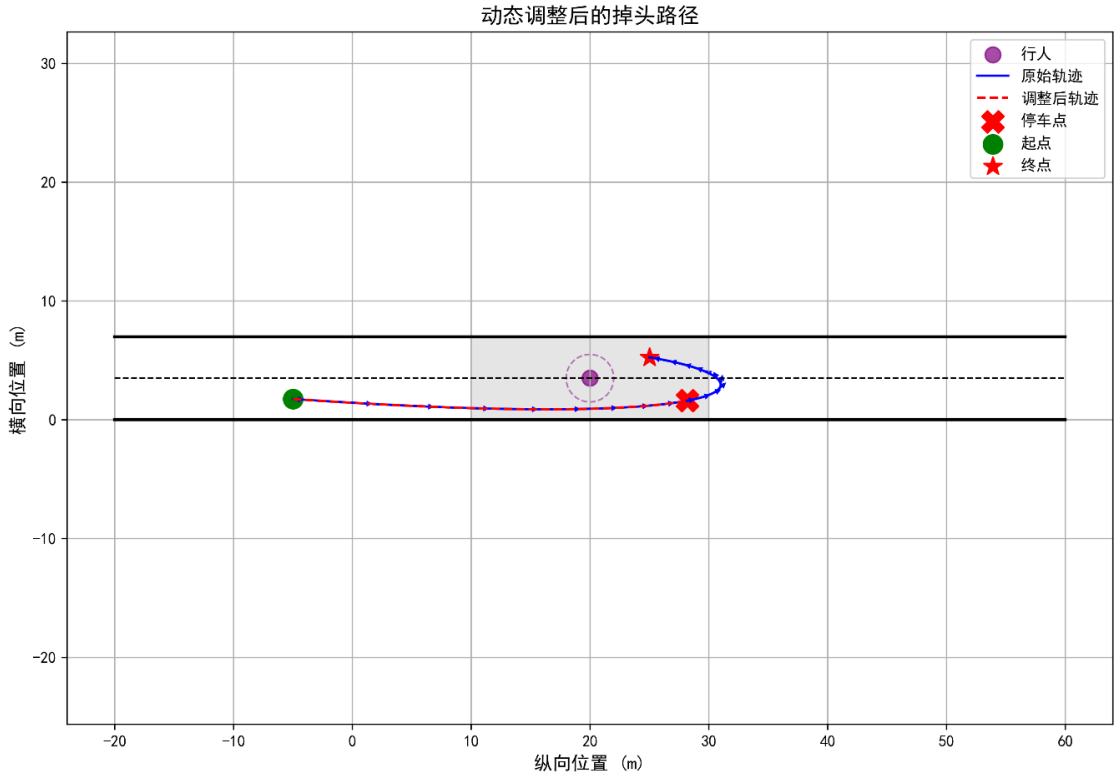

图 5.7 紧急情况路径曲线变化

由图 5.7 可直观的看出，若按原始路径掉头则极有可能与行人产生碰撞，因此控制 AV 车辆在掉头的过程中停止，则可有效避免与行人相撞。为更加清晰的看到其调整变化，给出动态调整结果如表 5.8 所示：

表 5.8 紧急情况指标结果

指标	原始值	调整后值	变化率(%)
平均横向加速度(m/s²)	0.478	0.478	0
最大横向加速度(m/s²)	0.701	0.701	0
平均速度(m/s)	6.82	6.23	-8.6
平均曲率(1/m)	0.0116	0.0116	0
总耗时(s)	7.77	3.61	-53.5

由表 5.8 中数据可知，该掉头路径没有进行任何变化，只是在掉头途中发生了急停情况，以避免与行人相撞。因此上述内容描述了两种不同的调整方法，一是对掉头路径进行细微的调整以此来避免与行人相撞情况，即避免突发情况的发生；二是在掉头的过程中及时停止掉头的操作，以此来避免危险的发生。虽然有不同的两种调整增方式，但是其不同的动作都取决于 AV 车辆对突发状况的风险评估，本文主要通过建立分层响应机制以及运用 MPC 算法来对不同的突发状况进行判断以及调整。

6 模型的分析与检验

6.1 模型分析

6.1.1 误差分析

1．轨迹预测误差

在运用卡尔曼滤波预测车辆位置时，预测过程中存在过程噪声，该过程中的噪声直接关联障碍物速度误差。通过将速度误差标准差纳入过程噪声协方差矩阵，仿真验证显示距离误差可控制在 0.3m 之内，远小于 1.5m 行人安全阈值，满足安全性判断精度需求，通过该过程噪声增加了模型的抗干扰能力。

2．路径拟合误差

贝塞尔曲线的参数化路径与实际行驶轨迹的偏差主要源于控制点的数量。通过设置四个关键控制点，使路径拟合误差小于 0.2m，满足道路边界的几何约束要求。

6.1.2 灵敏度分析

在问题二进行多目标优化时，运用了层次分析法计算每个目标函数的权重。在求解权重的过程中，有专家打分对重要性进行量化，但在不同情境下，对于安全性、效率以及舒适性的追求是不同的，因此每个目标的权重都是有所变化的、不是一成不变的。

于是，将通过改变每个目标函数的权重，来分析模型的灵敏度。通过对代码设定不同的权重，得出不同的优化结果，进行对比，来体现优化模型的灵敏度。对三个目标函数设置不同权重得到的指标的变化如表 6.1 所示。

表 6.1 指标变化

权重调整描述	权重变化详情	最小安全距离变化	总耗时变化	横向加速度波动变化
安全权重降低	ω_3 降低 0.06，ω_1 增加 0.03，ω_2 增加 0.03	-0.28m	-5.81s	-13.40%
舒适性权重提高	ω_2 增加 0.06，ω_1 减少 0.03，ω_3 减少 0.03	-0.08m	+4.98s	-32.00%
时间权重提高	ω_1 增加 0.06，ω_2 减少 0.03，ω_3 减少 0.03	-0.21m	-7.12s	13.50%

由表 6.1 中数据可知，若降低安全性的权重，即 ω_3 变为 0.577，则其最小安全距离将减少 0.28m，其总耗时将减少 5.81s，横向加速度波动减少 13.40%。由于安全性权重减少且总权重保持不变，则关于效率以及耗时的权重将增加。因此由于安全性权重降低，则最小安全距离将会减少，且因为更加看重耗时以及横向加速的波动变化，其他方面的数值也将会会减少。

若提高舒适性权重，则其安全距离将减少 0.08m，总耗时增加 4.98s，横向加速度波动减少 32%。由于舒适性权重提高，则横向加速度波动自然会有所降低，且横向加速度波动的变化也会直接影响到总耗时的变化，横向加速度波动越小那么 AV 在掉头过程中的速度也会变慢则总时长增大。而由于舒适性权重增大将会导致安全性权重减小，又因为安全性始终在掉头过程中占主要影响因素，则其安全距离将会有略微的减少。

若提高时间权重，则其安全距离将减少 0.21m、总耗时减少 7.12s、横向加速度波动增加 13.5%。由于时间权重的提高，则总耗时在目标函数中的占比变大，则在进行优化时会更加看重其总耗时的影响，则总耗时会减少。又由于时间权重增加，总耗时减小，则在转弯过程中 AV 的速度将会变大，其横向加速的波动也会加大。且因为总权重不变其中时间权重增加，则安全性权重将会减少，则在优化过程时其最小安全距

离将会减少。

根据灵敏度分析可知，模型对核心参数的波动具有较强容忍性，关键指标变化均在赛题约束范围内（如安全距离始终≥1.5m、横向加速度≤1.2m/s²），验证了模型设计的合理性。

6.2 模型验证

对于层次分析法确定的多目表函数的权重，每个目标的重要性可能会与层次分析开始定义的 pairwise 矩阵中的重要性不同，因此需要进行一致性检验，要计算其最大特征值、一致性指标以及随即一致性比率，且要随即一致性比率$CR < 0.1$。此时，则表明该权重通过了一致性检验，即该权重合理。因此，通过一致性检验可使层次分析法计算的权重更加合理，建立的模型更加有效。

7 模型的评价、改进与推广

7.1 模型的优点

首先，模型考虑安全性优先，通过卡尔曼滤波预测，有效处理障碍物速度误差，提升模型的鲁棒性和动态环境适应性。

其次，安全阈值量化科学，结合最小距离（行人 1.5m、车辆 2m）与碰撞时间（TTC），通过归一化安全窗口（0-1）实现决策可视化，避免单一指标的局限性。

然后，多目标优化逻辑清晰，权重分配合理。通过层次分析法（AHP）确定的权重（安全距离 0.627、舒适性 0.264、总耗时 0.109）符合实际交通场景中“安全优先、兼顾效率与舒适”的原则，且通过一致性检验（CR=0.0332<0.1），逻辑无矛盾。

此外，路径规划采用参数化曲线（贝塞尔曲线），确保位置、速度、加速度连续，横向加速度波动从优化前的 0.28 降至 0.0236，显著提升乘坐体验。

最后，考虑了工程实用性，仅依赖常规传感器（雷达、摄像头），便于实际情况下的操作和执行，算法复杂度低（每步计算≤0.1 秒），简单易懂且可实时部署。

7.2 模型的缺点

障碍物行为预测存在局限性，卡尔曼滤波基于线性运动假设，对突发行为（如行人突然折返、社会车辆急刹）的预测误差较大，未考虑障碍物之间的交互影响。

路径衔接处存在平滑性不足的缺点。贝塞尔曲线在掉头区入口与出口的衔接处存在轻微速度波动（±0.3m/s），虽满足舒适性阈值，但仍有优化空间。

忽略了车辆自身长度、宽度的影响，对车辆量的掉头过程产生误差，路径曲线会存在略微的差异。

7.3 模型的改进

1．提升障碍物行为预测精度。

引入交互感知模型，对社会车辆，结合历史变道数据预测变道概率；对行人，采用 LSTM 网络学习横穿意图（如停留位置、身体朝向），降低突发行为预测误差。

2．优化路径平滑性

采用更精确的曲线代替贝塞尔曲线，增强路径的平滑性。

7.4 模型的推广

1．场景扩展

可直接推广至城市支路（窄路场景）、园区内部道路等低速场景，通过调整最小转弯半径和安全距离阈值适配环境；经参数优化后，也可应用于无保护左转等类掉头场景。

2．商业落地

算法可集成至 L2+级自动驾驶系统的低速辅助模块，通过简化 MPC 预测步数适配车载嵌入式硬件，单帧计算耗时可控制在 0.05s 以内，降低工程部署成本。

3．智能交通协同

与交通信号灯系统联动，根据掉头相位时长动态调整效率权重——高峰期（流量＞1000 辆/小时）安全权重提升至 0.7，平峰期（流量＜500 辆/小时）效率权重提升至 0.15，实现车路协同优化。

参考文献

[1] 李霖. 掉头场景下自动驾驶车辆轨迹规划与跟踪控制研究[D]. 吉林大学, 2023.
[2] 陈淑琬, 赵鹏飞, 刘丹丹, 等. 2005-2021 年中国道路交通事故死亡趋势分析[J]. 疾病监测, 2025, 40(01): 133-137.
[3] 国务院印发《交通强国建设纲要》[J]. 建筑技术, 2019, 50(10): 1159.
[4] 李克强, 戴一凡, 李升波, 等. 智能网联汽车(ICV)技术的发展现状及趋势[J]. 汽车安全与节能学报, 2017, 8(01): 1-14.
[5] 严明. 自动驾驶车辆掉头路径规划与跟踪控制研究[D]. 扬州大学, 2023.
[6] 田大新,肖啸,周建山.AI 驱动的自动驾驶汽车轨迹预测方法综述[J/OL].交通运输系统工程与信息,1-43[2025-08-18].
[7] TETOU L M K. A comparative analysis of object tracking methods[D]. Lappeenranta-Lahti University of Technology LUT, 2023.
[8] 张爱琳，张秀英，许有俊，等.基于改进 AHP-熵值法的城市综合管廊施工进度风险模糊综合评价研究[J].建筑技术，2017,48(09):922-926.

附录

附录 1
介绍：支撑材料的文件列表
1. 任务二优化前掉头路径结果表格.csv 2. 任务二优化前掉头路径可视化.png 3. 任务二优化前掉头路径完整数据.npz 4. 任务二优化后掉头路径结果表格.csv 5. 任务二优化后掉头路径可视化.png 6. 任务二优化后掉头路径完整数据.csv 7. 权重灵敏度分析结果.csv 8. 任务三急停动态调整结果表格.csv 9. 任务三急停动态调整后掉头路径数据.npz 10. 任务三急停动态调整结果可视化.png 11. 任务三急停动态调整模拟日志.txt 12. 任务三急停原始掉头路径数据.npz 13. 任务三路径微调结果表格.csv 14. 任务三路径微调后掉头路径数据.npz 15. 任务三路径微调结果可视化.png 16. 任务三路径微调模拟日志.txt 17. 任务三路径微调原始掉头数据.npz 18. Task1.m 19. Task2.py 20. 灵敏度分析.py 21. Task3Stop.py 22. Task3Fine-tuning.py

附录 2
介绍：该代码是 matlab 语言编写的，实现了任务一中自动驾驶车辆在复杂道路环境中的掉头决策与运动规划系统。

```
for k = 1:steps
    currentObstacles = getCurrentObstacles(obstacles, k);

    [predictedTrajectories, uncertainties] = predictObstacleTrajectories(...
        obstacleHistory, currentObstacles, dt, k);

    [safetyWindow, minDistances] = calculateSafetyWindow(...
        avState, scene, av, currentObstacles, predictedTrajectories, uncertainties, dt);
    safetyWindows(k) = safetyWindow;

    [decision, qValues] = makeTurnDecision(avState, safetyWindow, minDistances);
```

```
    decisionLog(k) = decision;

    if decision == 1 && avState.turnCompleted == false
        [path, speedProfile, turningAngles] = planOptimalPath(...
            avState, scene, currentObstacles, predictedTrajectories, dt);
        avState = updateAVState(avState, path, speedProfile, turningAngles, dt);
        pathPlan{k} = struct('x', path(:,1), 'y', path(:,2), ...
            'v', speedProfile, 'theta', turningAngles);
    else
        avState = maintainOrSlowDown(avState, dt);
    end

    obstacleHistory = updateObstacleHistory(obstacleHistory, currentObstacles, k);
end

function [decision, qValues] = makeTurnDecision(avState, safetyWindow, minDistances)
    decision = 0;
    if ~isempty(minDistances)
        minDistance = min(minDistances);
    else
        minDistance = Inf;
    end

    cond1 = safetyWindow > 0.7;
    cond2 = minDistance > 2.5;
    if cond1 && cond2
        decision = 1;
    end
    qValues = [safetyWindow, minDistance];
end
```

附录 3

介绍：该代码是 python 语言编写的，实现了任务二中基于贝塞尔曲线的自动驾驶车辆双车道掉头路径规划与优化系统。

```
class BezierPathPlanner:
    def __init__(self, scene_params, av_params, obstacles=None):
        self.scene = scene_params
        self.av = av_params
        self.obstacles = obstacles if obstacles else []
        self.weights = calculate_ahp_weights()
```

```
    def generate_bezier_path(self, params):

        x0, y0 = self.av.initial_x, self.av.initial_y
        x3, y3 = self.av.final_x, self.av.final_y
        x1, y1 = 5.0 + params[0], 6.0 + params[1]
        x2, y2 = 15.0 + params[2], 6.0 + params[3]
        control_points = np.array([[x0, y0], [x1, y1], [x2, y2], [x3, y3]])
        t = np.linspace(0, 1, 100)
        path = np.zeros((len(t), 2))

        for i, t_val in enumerate(t):
            path[i] = (
                (1-t_val)**3 * control_points[0] +
                3*(1-t_val)**2*t_val * control_points[1] +
                3*(1-t_val)*t_val**2 * control_points[2] +
                t_val**3 * control_points[3]
            )
        return t, path[:, 0], path[:, 1], control_points

    def objective_function(self, params):
        total_time = 10.0 + params[4]
        t_bezier, x, y, _ = self.generate_bezier_path(params[:4])
        t = np.linspace(0, total_time, len(x))

        J1 = total_time
        lat_acc = self.calculate_lateral_acceleration(y, t)
        J2 = quad(lambda ti: np.interp(ti, t, lat_acc)**2, 0, total_time)[0]
        min_dist = self.calculate_min_distance_to_obstacles(x, y)
        J3 = -min_dist

        return 0.2*J1 + 0.2*J2 + 0.6*J3

    def constraint_functions(self, params):

        total_time = 10.0 + params[4]
        t_bezier, x, y, _ = self.generate_bezier_path(params[:4])

        curvature = self.calculate_curvature(x, y)
        lat_acc = self.calculate_lateral_acceleration(y, np.linspace(0, total_time, len(x)))
        min_radius = 1/np.max(np.abs(curvature)) if np.max(np.abs(curvature)) > 0 else
np.inf
        return [
            np.max(np.abs(lat_acc)) - self.av.max_lat_acc,
```

```
                self.av.min_turn_radius - min_radius,
                np.min(y) - 0.5,
                (self.scene.road_width - 0.5) - np.max(y),
                self.calculate_min_distance_to_obstacles(x, y) - 2.0
            ]
    def optimize_path(self):
        initial_params = [0.0, 0.0, 0.0, 0.0, 0.0]
        bounds = [(-2.0, 2.0)]*4 + [(0.0, 10.0)]

        result = minimize(
            self.objective_function,
            initial_params,
            method='COBYLA',
            constraints={'type': 'ineq', 'fun': self.constraint_functions},
            options={'maxiter': 2000}
        )

        t_bezier, x, y, _ = self.generate_bezier_path(result.x[:4])
        total_time = 10.0 + result.x[4]
        t = np.linspace(0, total_time, len(x))
        curvature = self.calculate_curvature(x, y)
        return {
            't': t, 'x': x, 'y': y,
            'speed': self.generate_speed_profile(x, curvature, total_time),
            'lat_acc': self.calculate_lateral_acceleration(y, t),
            'curvature': curvature
        }

def main():

    scene = SceneParams()
    av = AVParams()
    obstacles = [Obstacle(35, 1.75, 'vehicle')]

    planner = BezierPathPlanner(scene, av, obstacles)
    optimized_path = planner.optimize_path()

    visualize_results(planner, optimized_path)
    generate_result_table(planner, optimized_path)
```

附录 4

介绍：该代码是 python 语言编写的，实现了任务三中自动驾驶车辆掉头场景中的动态障碍物响应与路径调整系统。

```
class TTCCalculator:
    @staticmethod
    def calculate_ttc(av_state, obstacle):
        rel_pos_x = obstacle.x - av_state[0]
        rel_pos_y = obstacle.y - av_state[1]
        rel_vel_x = obstacle.vx - av_state[2]
        rel_vel_y = obstacle.vy - av_state[3]

        rel_dist = np.sqrt(rel_pos_x**2 + rel_pos_y**2)
        if rel_dist < obstacle.safe_distance + obstacle.size:
            return 0.0
        rel_speed_sq = rel_vel_x**2 + rel_vel_y**2
        if rel_speed_sq < 1e-6:
            return np.inf
        return rel_dist / np.sqrt(rel_speed_sq)
class DynamicAdjustmentHandler:
    def __init__(self, av_params):
        self.av = av_params
        self.mpc = MPCController(av_params)
        self.response_level = 0
        self.is_interrupted = False
        self.终止_index = None

    def determine_response_level(self, min_ttc, current_distance, safe_distance):

        if min_ttc <= 1.5 or current_distance < safe_distance:
            return 3
        elif 1.5 < min_ttc <= 3.0:
            return 2
        elif 3.0 < min_ttc <= 10.0:
            return 1
        else:
            return 0

    def adjust_speed_profile(self, original_profile, response_level, current_idx):

        adjusted_profile = original_profile.copy()
        if response_level == 2:
            decel = self.av.max_long_decel * 0.6
            for i in range(current_idx, len(adjusted_profile)):
                time_step = (i - current_idx) * self.av.control_cycle
```

```
                adjusted_speed = adjusted_profile[current_idx] + decel * time_step
                adjusted_speed = max(adjusted_speed, self.av.final_speed * 0.5)
                adjusted_profile[i] = adjusted_speed

        elif response_level == 3:
            decel = self.av.max_long_decel
            for i in range(current_idx, len(adjusted_profile)):
                time_step = (i - current_idx) * self.av.control_cycle
                adjusted_speed = adjusted_profile[current_idx] + decel * time_step
                adjusted_speed = max(adjusted_speed, 0)
                adjusted_profile[i] = adjusted_speed

                if adjusted_speed <= 0 and self.终止_index is None:
                    self.终止_index = i
                    adjusted_profile[i:] = 0
                    break
        return adjusted_profile

    def handle_dynamic_adjustment(self, current_state, reference_path, path_data, obstacles):
        min_ttc = np.inf
        closest_obs = None
        for obs in obstacles:
            ttc = TTCCalculator.calculate_ttc(current_state, obs)
            if ttc < min_ttc:
                min_ttc = ttc
                closest_obs = obs
        if closest_obs:
            current_distance = np.sqrt((closest_obs.x - current_state[0])**2 +
                                       (closest_obs.y - current_state[1])**2)
            self.response_level = self.determine_response_level(
                min_ttc, current_distance, closest_obs.safe_distance + closest_obs.size)
        else:
            self.response_level = 0
            return path_data,
        if self.response_level == 1:
            steering_angle, acceleration = self.mpc.optimize_controls(current_state, reference_path, obstacles)
            return path_data, f"Level 1 - MPC 微调，转向角: {steering_angle:.3f}rad"

        elif self.response_level == 2:
            current_idx = np.argmin(np.sqrt((path_data['x'] - current_state[0])**2 +
                                            (path_data['y'] -
```

```
current_state[1])**2))
                adjusted_speed = self.adjust_speed_profile(path_data['speed'], 2,
current_idx)
                path_data['speed'] = adjusted_speed
                return path_data, f"Level 2 - 减速避让，TTC: {min_ttc:.2f}s"

            elif self.response_level == 3:
                current_idx = np.argmin(np.sqrt((path_data['x'] - current_state[0])**2 +
                                                (path_data['y'] -
current_state[1])**2))
                adjusted_speed = self.adjust_speed_profile(path_data['speed'], 3,
current_idx)
                path_data['speed'] = adjusted_speed
                self.is_interrupted = True
                return path_data, f"Level 3 - 紧急制动，TTC: {min_ttc:.2f}s"
class SimplePathGenerator:
    def simulate_dynamic_obstacles(self, path_data, 突发障碍物=None):
        simulation_log = []
        current_obstacles = []
        adjusted_path_data = path_data.copy()

        for i in range(len(path_data['t'])):
            for obs in current_obstacles:
                obs.update_position(self.av.control_cycle)
            if i == len(path_data['t']) // 3 and 突发障碍物:
                current_obstacles.extend(突发障碍物)
                simulation_log.append(f"检测到突发障碍物: {[obs.type for obs in
突发障碍物]}")
            current_state = [path_data['x'][i], path_data['y'][i],
                             path_data['heading'][i], path_data['speed'][i]]
            adjusted_path_data, action =
self.dynamic_handler.handle_dynamic_adjustment(
                current_state, adjusted_path_data, adjusted_path_data,
current_obstacles)

            if action != "无响应":
                simulation_log.append(f"时间 {path_data['t'][i]:.1f}s: {action}")

            if self.dynamic_handler.is_interrupted and self.dynamic_handler.终止
_index is not None:
                simulation_log.append("已安全停车，等待障碍物离开")
                break
                return adjusted_path_data, simulation_log
```